HW-1, Roll No. 2019115001

“Bitcoin: A Peer-to-Peer Electronic Cash System” by Satoshi Nakamoto

A Paper Summary

1. The problem, at scratch.

KEYTERMS: *double-spending, trust-based models, non-reversible transactions, de-centralized systems, blockchain*

1.1. Digital money is trivially reversible, if not mediated.

Imagine money is just a file on my computer i.e. a string of digital tokens (not far off from what electronic payments involve). As soon as money becomes a digital entity, we run into a problem - **files can be copied, unlike physical cash** which can only exist at point A or point B only as constrained by the laws of physics. Say I email Rs 500 as a file to one of the TAs of this course for the purpose of bribing them into grading me an A on this homework, guess what? I still have my copy. So on the off chance that the TA does not grade me an A, I can email the same Rs 500 to as many TAs as there are in the course as insurance without having to spend anything extra to accommodate my outrageous and dishonest behaviour. This is called **double spending**.

Double-spending represents the act of spending the same digital asset more than once, and as highlighted by the above hypothetical - it can compromise the security of the entire network if the payment system does not have infrastructure built-in to accommodate mechanisms that actively protect their users - payers and payees alike. Traditionally, this is fixed using **trust-based models** where financial institutions act as mediators between payer and payee for each and every transaction that happens on their network. Each coin that is part of a transaction goes through a **mint** or a trusted central authority (traditional banks for example), and is verified to ensure it has not double-spent earlier. Once validated, the coin is reissued by the mint and only coins issued by the mint are trusted to not be double-spent. Hence the tag "trust-based". (Section-2, Nakamoto)

1.2. Author's critique of trust-based models.

As the author notes in the first section, commerce today relies almost exclusively on financial institutions to act as third-parties to process electronic payments. However trust-based models have several weaknesses that Nakamoto highlights:

1. Transactions cannot be completely **non-reversible**

   Disputes in electronic transactions cannot be avoided, and are quite common - your credit card can get stolen and misused for unauthorized transactions, online merchants may charge for a product and never send it to the buyer, double-billing, the list goes on. Without the ability to reverse transactions, banks cannot resolve any of these problems. Though this can be seen as a "feature, not a bug", **reversibility comes with real cost** (Section-1, Nakamoto). Every time a dispute requires resolution, it costs the bank: money, time and staff. This is precisely why banks charge "processing fees" on every card swipe, even transactions that don't lead to disputes; these fees are pooled into funds the bank uses for dispute-resolution among other things. One might argue that service fees are ubiquitous and span across almost every business so it's only fair the bank also charges one. However in the case of electronic payments, **casual micro-transactions can become bloated and thus impractical.**

   The author notes that non-reversible transactions would not only protect buyers but sellers as well (Section-1, Nakamoto). **Reversibility is usually framed as a protection for consumers but it cuts both ways - it exposes risk for sellers too.** Often, buyers receive goods and then falsely dispute the charge to get their money back. Since transactions can never be guaranteed to be final, merchants and businesses build defences and hold on to more information than necessary about buyers to protect purchases, adding friction to every transaction.

Nakamoto looks at how physical currency bypasses all such uncertainties and costs, since **cash is inherently non-reversible**: a cash transaction is final the instant it happens (no one expects to reverse a cash sale, barring returns/refunds negotiated b/w the parties themselves of course). Nakamoto treats the finality of cash payments as the desirable baseline for electronic systems but notes "there is no mechanism in place to make payments over a communications channel without a trusted party" (Section-1, Nakamoto).

2. Centralized control exercised by financial institutions

   Every transaction no matter how small or private must make a round-trip through a central authority for validation - whether it involves money that has been duplicated earlier or not. **The fate of the entire money system depends on the company that controls the mint**. (Section-2, Nakamoto).

   Nakamoto proposes an alternate system where control is **decentralized** over a network of peers that maintain a single network-wide, *computationally-secure*, public ledger and access to this ledger is distributed across nodes so that each has their own copy. This ledger is referred to as the `**blockchain**`.

   NOTE: This is a highly abstracted description of BitCoin from what I have understood after a fourth pass of the paper. We will take a closer look at how the system works, and the complex mechanisms that underlie it soon.

2. BitCoin

KEYTERMS: *hashfunctions, cryptographic signatures, proof-of-work, computational work, merkle trees, gambler's ruin*

2.1. The anatomy of a bitcoin transaction

Nakamoto defines a coin as literally a chain of digital signatures, nothing more (Section-2, Nakamoto). Every time a coin exchanges ownership, the payer signs a hash of the previous transaction along with the payee's public key, and this signature is attached to the end of the coin. Hence, **a *bitcoin* isn't a static object sitting in a wallet : it's a growing history of every hop it's made from person to person, and each hop is cryptographically signed**.

Every peer holds a pair of keys: a public key that every node can see and a private key that only is never shared to the network. When Owner 1 transfers a coin to Owner 2, they sign using their private key. Owner 2 can use the public key of Owner 1 and the signature on the transaction and verify whether the transaction was “honest” or not - signature mismatch, forgery or tampering are examples of dishonest transactions. These validations are done at the node level i.e. a node can verify the chain of signatures back to when the coin was first created at, and confirm the coin has changed hands honestly each time.

However the double-spending problem has yet to be solved. **The chain of signatures tells Owner 2 nothing about whether Owner 1 also signed this same coin over to someone else 3 minutes earlier, since both transactions would have valid signatures. Verifying transaction signatures only proves the chain of custody is legitimate, not that it's unique**. So we're back to **double-spending**.

“the only way to confirm the absence of a transaction is to be aware of all transactions”
(Section-2, Nakamoto)

As discussed above and by the author as well, traditional systems used to solve this by tracking every transaction and creating mechanisms that could process repeats instantly. **Nakamoto's whole project boils down to replacing the trust-based model of electronic payments with a proof-of-work based model while getting that same guarantee - “has this coin been spent before?” - without a mint**. **If transactions are verified through querying the chain of signatures, the only way to confirm an absence of a transaction -- a necessary condition if the coin was not double-spent -- is to be aware of all transactions.** This requires the distributed system to have a *time-stamp server* (Section-3, Nakamoto) in play that publishes transactions that are validated by node as ground-truth that *that* transaction actually happened..

2.2. The Mechanism of Consensus: Proof-of-Work

Since there is no central bank to act as a referee, the network needs a way for nodes to agree on which transactions happened and in what. This is referred to as ***consensus*** : distributed nodes in a network rely on protocols and converge towards agreement on a single history. **The protocol for reaching consensus in BitCoin’s distributed network is done through a consensus algorithm called *proof-of-work*** (Section-4, Nakamoto). It is effectively a competitive puzzle-solving race. Participants called *miners* bundle a group of new transactions into a "*block”*. To make that block official, the miner has to solve a computationally expensive mathematical puzzle -- specifically, finding a number called a *nonce* (an integer value part of the block header) that when hashed using a *cryptographic cost function* such as SHA256, with the block's data, produces a desired result according to the requirements of the network. starting with

a specific number of zeros in the leading bits -- usually the first 30 bits must be set to 0. (Section-4, Nakamoto).

Hash functions like SHA256 can take any input of variable length and turn it into a fixed-length, effectively random looking output. The mathematics of cryptographic hash functions (as outlined in the HashCash paper referenced by Nakamoto) make it so that these functions are **easy to verify, yet virtually impossible to decipher** (Section-4, Nakamoto; Back, 2002) -- there is absolutely no way of reverse-engineering the key using its hashed value (there doesn't exist any formal proof that can verify this claim, yet the majority of security that underlies modern networks). And yet, once the key is known, verification takes a few milliseconds since you only need to run the function and check the bits of the hashed output. Additionally, checking even a single character in the block data (say, changing "Bob paid Alice 1 BTC" to "Bob paid Alice 10 BTC" completely changes the value of the hash. This is called the **Avalanche effect** and it is an inherent feature of cryptographic cost functions. **BitCoin's security relies on these features.**

Miners compete to brute-force block validations by using specialized hardware called ASIC miners that can make between 300 trillion and 1.16 quadrillion of guesses per second. Once a lucky miner lands a valid hash according to the network's security requirements -- first N bits must be set to 0 (difficulty exponentially increases with the value of N) -- it broadcasts the finished block. Other nodes check it within milliseconds, add it to their copy of the chain, and the winning miner takes a *block reward* - the newly issued BTC coins the network mints (Section-6, Nakamoto). **This computational work is what secures the network. It establishes a "one-CPU-one-vote" system, ensuring that the majority decision is backed by the most computing power** (Section-4, Nakamoto)**.** Since each block contains the hash of the previous one, changing a single transaction would require redoing the work for that block and every single block that comes after it, which becomes physically impossible very quickly.

2.3. Block Structure & Space Saving

"*It is possible to run verifications without running a full network node*" (Section-8, Nakamoto)

The blockchain, visually, is just a chain of blocks - each one referencing its predecessor via hashes. To keep the ledger from bloating past what a regular machine can hold, **transactions in a block are hashed pairwise into a Merkle Tree until a single "root hash" remains, and only that root goes into the header** (Section-7, Nakamoto; Merkle, 1980)**.** So the block header stays

tiny (~80 bytes with no transactions in it), and this is exactly what lets nodes verify a payment without downloading the entire multi-gigabyte chain, just against the headers and a Merkle branch (Section-7 & 8, Nakamoto).

2.3. Security & Privacy

Security reduces to a *Gambler's Ruin* problem: an attacker rewriting history is racing the honest network to build a longer chain (Section-11, Nakamoto; Feller, 1957). **As long as honest miners hold the majority of compute power, the attacker's odds of catching up fall exponentially with every block added on top**, which is why merchants conventionally wait for ~6 confirmations before treating a large payment as final (Section-11, Nakamoto). Conflicting histories in the case of a malicious node forging transactions can only last for so long (Section-5, Nakamoto).

Privacy flips the usual model - transactions stay public, but identities don't (Section-10, Nakamoto), since public keys aren't tied to real names. In this manner, Bitcoin as a platform is similar to how stock exchanges work - trade sizes are published without mentioning the parties involved. The system stays secure through the use of asymmetric cryptographic key pairs as part of the validation itself without sacrificing user privacy. The author suggests that a fresh key can be used per transaction makes it even harder for an observer to link a string of payments back to one person, especially when signatures get buried deep within hashes of hashes (Section-10, Nakamoto).

3. Conclusion

Ultimately, Bitcoin replaces the need for a trusted third party with cryptographic proof. By combining digital signatures to prove ownership and Proof-of-Work to prove the timeline of events, it creates a decentralized system where the rules are enforced by math rather than institutions. It’s an elegant solution that makes it more profitable for participants to support the network than to try and cheat it (Section-12, Nakamoto).

References

Back, A. (2002). *Hashcash – a denial of service counter-measure*. http://www.hashcash.org/papers/hashcash.pdf

Feller, W. (1957). *An introduction to probability theory and its applications*. John Wiley & Sons.

Merkle, R. C. (1980). Protocols for public key cryptosystems. In *Proceedings of the 1980 IEEE Symposium on Security and Privacy* (pp. 122–134). IEEE Computer Society.

Nakamoto, S. (2008). *Bitcoin: A peer-to-peer electronic cash system*. https://bitcoin.org/bitcoin.pdf

----------xxxx----------